

Bài tập thực hành Lập trình API Hệ thống quản lý Facebook Page phân tán

Học phần: Lập trình API

Ngày 9 tháng 5 năm 2026

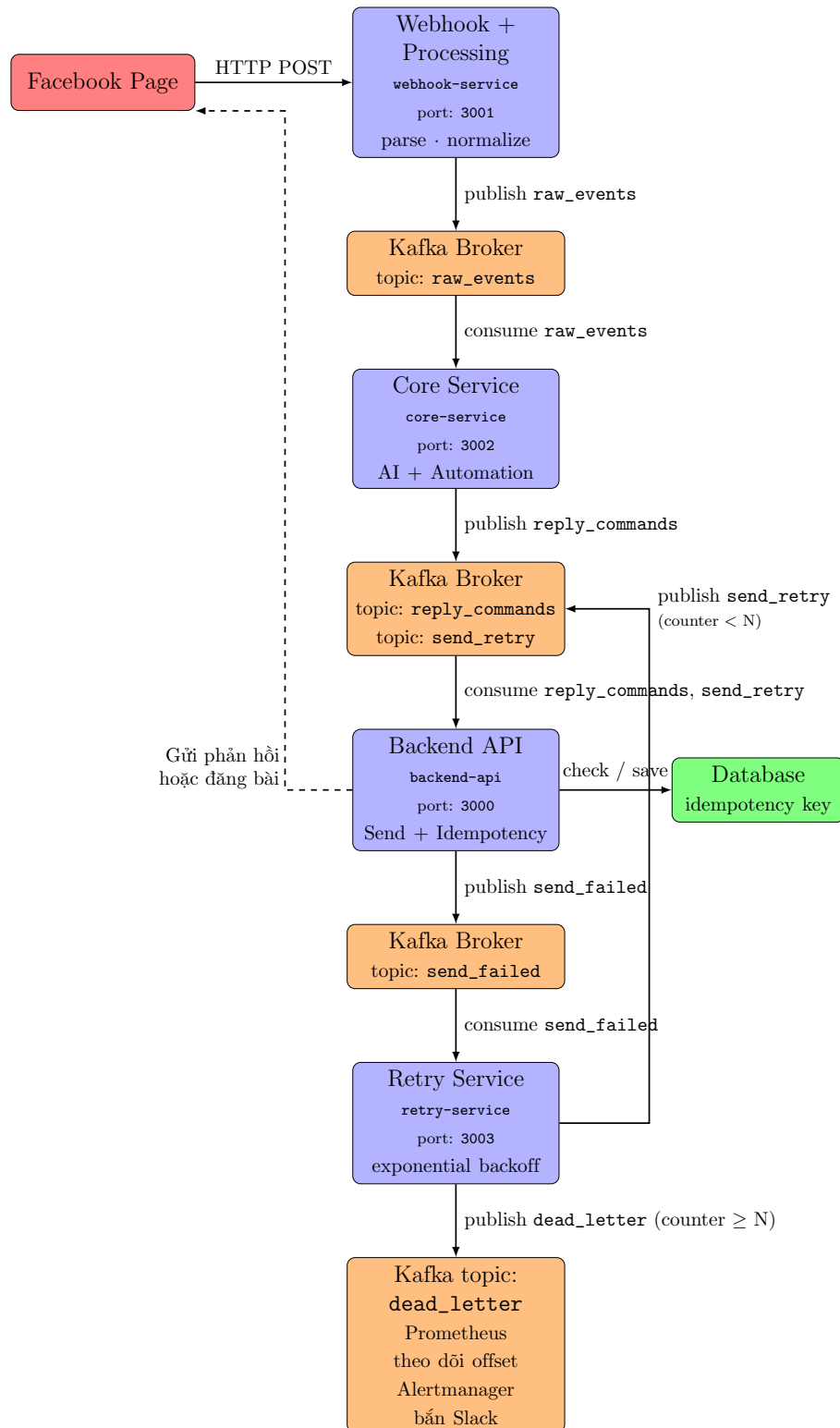
1 Tổng quan

Bài tập này yêu cầu sinh viên thiết kế và triển khai một hệ thống phân tán kết nối với Facebook Graph API, xử lý sự kiện theo thời gian thực, truyền dữ liệu qua Kafka và phân tích cảm xúc bằng AI API.

Giải thích: Mục tiêu của bài không chỉ là gọi được API, mà còn là hiểu cách nhiều service nhỏ phối hợp với nhau trong một hệ thống thực tế, thấy được luồng dữ liệu đi từ Facebook, qua các service xử lý, rồi quay trở lại hệ thống để phản hồi hoặc lưu trữ.

2 Kiến trúc hệ thống

2.1 Sơ đồ kiến trúc



2.2 Mô tả các service

- **Facebook Page:** Khi có người dùng bình luận hoặc nhắn tin, Facebook gửi HTTP POST đến Webhook endpoint của hệ thống.
- **Webhook + Processing (service webhook-service, port 3001):** Nhận event từ Facebook. Service này xác thực chữ ký HMAC-SHA256 để đảm bảo request đến từ Facebook thật, parse payload JSON, normalize về schema chuẩn nội bộ (comment và message có format khác nhau nhưng sau bước này ra cùng một cấu trúc), rồi publish vào topic `raw_events`. Service phải trả 200 OK cho Facebook càng nhanh càng tốt để tránh bị retry.
- **Kafka Broker:** Hạ tầng truyền thông điệp trung tâm. Mọi giao tiếp nội bộ giữa các service đều đi qua Kafka. Hệ thống sử dụng các topic sau:
 - `raw_events`: Webhook + Processing publish, Core Service consume.
 - `reply_commands`: Core Service publish, Backend API consume.
 - `send_retry`: Retry Service publish, Backend API consume.
 - `send_failed`: Backend API publish, Retry Service consume.
 - `dead_letter`: Retry Service publish khi hết số lần thử. Không có consumer. Prometheus theo dõi offset để phát hiện message mới và Alertmanager bắn cảnh báo ngay khi có message vào topic này.
- **Core Service (service core-service, port 3002):** Consume topic `raw_events` và xử lý theo hai bước tuần tự. Bước AI gọi mô hình ngôn ngữ để phân loại intent (hỏi giá, khiếu nại, spam...) và phân tích sentiment (tích cực, trung tính, tiêu cực). Sinh viên có thể chọn một AI API phù hợp như **OpenAI**, **Gemini**, **Claude**, **Grok** hoặc dịch vụ tương đương, miễn là mô tả rõ đầu vào, đầu ra, chi phí sử dụng và phương án fallback khi API chậm hoặc lỗi. Bước Automation áp dụng rule engine để quyết định hành động: tự động reply, ẩn bình luận, đưa vào hàng chờ duyệt thủ công hoặc đánh dấu blacklist nội bộ. Với trường hợp tái phạm nhiều lần, nên để quản trị viên block thủ công trên Facebook Page; chỉ thử block qua API nếu nhóm đã tự kiểm chứng endpoint đó hoạt động ổn định. Kết quả publish vào topic `reply_commands`.
- **Backend API (service backend-api, port 3000):** Consume topic `reply_commands` và `send_retry`. Đây là service duy nhất được phép gọi Facebook Graph API. Trước khi gửi, service kiểm tra idempotency key trong Database để tránh gửi reply trùng khi Kafka redeliver cùng một message. Nếu gọi Facebook thành công thì lưu key lại, nếu thất bại thì publish `send_failed` để Retry Service xử lý. Service này cũng expose REST API cho dashboard quản trị.
- **Database:** Lưu idempotency key của từng `command_id` đã được xử lý, đảm bảo mỗi reply chỉ được gửi đúng một lần dù message được consume nhiều lần.
- **Retry Service (service retry-service, port 3003):** Consume topic `send_failed`. Đọc `retry_count` trong message, tính thời gian chờ theo exponential backoff ($1s \times 2^{\text{retry_count}}$). Nếu chưa đến ngưỡng tối đa N lần thì publish lại vào `send_retry` để Backend API thử gửi lại. Nếu đã đến ngưỡng thì publish vào `dead_letter` và dừng retry.

- **Dead Letter Queue (dead_letter topic):** Không phải service mà là một Kafka topic. Lưu toàn bộ message thất bại sau khi đã retry hết số lần cho phép. Prometheus theo dõi offset của topic này, khi offset tăng (có message mới), Alertmanager lập tức gửi cảnh báo đến Slack hoặc Email của nhóm vận hành. Admin có thể dùng Kafka UI (Kafdrop hoặc Redpanda Console) để xem nội dung message và xử lý thủ công.

2.3 Quy ước đặt tên service và port triển khai

- **backend-api:** Port 3000, expose REST API cho dashboard quản trị và là service duy nhất gọi Facebook Graph API.
- **webhook-service:** Port 3001, nhận webhook từ Facebook, verify chữ ký và đẩy event vào Kafka.
- **core-service:** Port 3002, xử lý AI, sentiment và automation rule, có thể expose health check.
- **retry-service:** Port 3003, chịu trách nhiệm retry, có thể mở endpoint health check hoặc metrics.

2.4 Giao tiếp giữa các service

- **Mọi giao tiếp nội bộ đều qua Kafka:** Các service không gọi nhau trực tiếp bằng HTTP. Toàn bộ dữ liệu trao đổi nội bộ đi qua Kafka topic tương ứng.
- **Chỉ Backend API được gọi Facebook Graph API:** Webhook + Processing chỉ nhận event vào. Backend API là nơi duy nhất gọi ra Facebook để reply, ẩn bình luận hoặc đăng bài.
- **Idempotency bắt buộc:** Mọi consumer phải xử lý an toàn khi nhận cùng một message nhiều lần. Backend API dùng idempotency key trong Database, các service khác dùng event_id để dedup.
- **Retry có giới hạn:** Retry Service thực hiện tối đa N lần với exponential backoff. Message vượt ngưỡng được chuyển sang dead_letter thay vì retry vô hạn.

2.5 Chuẩn message giữa các service

Sinh viên cần tự định nghĩa message structure: Đề bài đã quy định topic, luồng xử lý và một số trường nghiệp vụ quan trọng như event_id, command_id và retry_count, nhưng chưa khóa cứng schema JSON cụ thể cho từng topic. Vì vậy, mỗi nhóm cần tự thiết kế message structure thống nhất giữa producer và consumer, sau đó mô tả rõ trong báo cáo hoặc tài liệu kỹ thuật. Trong phạm vi bài này, nhóm có thể thống nhất dùng JSON làm định dạng message giữa các service vì dễ debug, dễ mở rộng và phù hợp với payload từ webhook Facebook.

Nguyên tắc thiết kế tối thiểu:

- Mỗi message nên có schema_version để thuận tiện mở rộng về sau.

- Các field định danh như `event_id`, `command_id`, `comment_id`, `page_id` cần rõ ràng và ổn định.
- Timestamp nên dùng định dạng ISO 8601, ví dụ 2026-04-26T09:30:00Z.
- Field phục vụ retry như `retry_count`, `last_error`, `next_retry_at` nên có mặt trong các topic liên quan đến lỗi và retry.
- Cần phân biệt payload gốc từ Facebook với payload nội bộ đã normalize để các service phía sau không phụ thuộc trực tiếp vào format webhook ban đầu.

Ví dụ gợi ý cho một số topic:

- `raw_events`: chứa dữ liệu đã normalize từ webhook, ví dụ gồm `event_id`, `event_type`, `page_id`, `post_id`, `comment_id`, `user_id`, `message`, `created_at`.
- `reply_commands`: chứa quyết định từ Core Service, ví dụ gồm `command_id`, `event_id`, `action`, `target`, `reply_text`, `intent`, `sentiment`, `created_at`.
- `send_failed` và `send_retry`: nên có `command_id`, `event_id`, `retry_count`, `last_error` và payload cần gửi lại để Backend API có thể retry an toàn.
- `dead_letter`: nên giữ gần như đầy đủ message lỗi cuối cùng để nhóm vận hành có thể debug và xử lý thủ công.

Ví dụ JSON minh họa:

```
raw_events
{
  "schema_version": 1,
  "event_id": "evt_001",
  "event_type": "comment_created",
  "source": "facebook",
  "page_id": "123456789",
  "post_id": "post_001",
  "comment_id": "cmt_001",
  "user_id": "user_001",
  "message": "Shop oi gia bao nhieu?",
  "created_at": "2026-04-26T09:30:00Z"
}

reply_commands
{
  "schema_version": 1,
  "command_id": "cmd_001",
  "event_id": "evt_001",
  "action": "reply",
  "target": {
    "page_id": "123456789",
    "comment_id": "cmt_001"
  },
}
```

```

    "reply_text": "Da shop da gui thông tin chi tiết qua inbox.",
    "intent": "ask_price",
    "sentiment": "neutral",
    "created_at": "2026-04-26T09:31:00Z"
}

send_failed / send_retry

{
  "schema_version": 1,
  "command_id": "cmd_001",
  "event_id": "evt_001",
  "retry_count": 1,
  "last_error": "Facebook API timeout",
  "next_retry_at": "2026-04-26T09:31:05Z",
  "payload": {
    "action": "reply",
    "reply_text": "Da shop da gui thông tin chi tiết qua inbox."
  }
}

dead_letter

{
  "schema_version": 1,
  "command_id": "cmd_001",
  "event_id": "evt_001",
  "retry_count": 3,
  "failed_at": "2026-04-26T09:33:00Z",
  "final_error": "Facebook API timeout after maximum retries",
  "original_topic": "send_failed",
  "payload": {
    "action": "reply",
    "target": {
      "page_id": "123456789",
      "comment_id": "cmt_001"
    },
    "reply_text": "Da shop da gui thông tin chi tiết qua inbox."
  }
}

```

Lưu ý: Không yêu cầu mọi sinh viên dùng đúng một schema duy nhất, có thể thiết kế message structure tùy ý nhưng phải hợp lý, nhất quán, dễ mở rộng và hỗ trợ tốt cho dedup, idempotency, retry và giám sát hay không.

2.6 Sinh viên cần thực hiện những bước gì?

1. Tạo Facebook App, Facebook Page và cấu hình các quyền cần thiết để làm việc với Graph API và Webhooks.

2. Xây dựng Backend API làm lớp trung gian giữa frontend và Facebook Graph API, không cho frontend gọi trực tiếp tới Facebook.
3. Cài đặt Webhook + Processing Endpoint, hoàn thành bước verify webhook, xác thực HMAC-SHA256 và đăng ký nhận các sự kiện bình luận từ Facebook.
4. Thiết kế Kafka topic, producer và consumer để chuyển sự kiện từ webhook sang các service xử lý nội bộ.
5. Xây dựng Core Service tích hợp AI phân loại intent, phân tích sentiment và Automation Logic ra quyết định phản hồi.
6. Xây dựng cơ chế xử lý lỗi hoàn chỉnh: Retry Service với exponential backoff, Dead Letter Queue và cảnh báo qua Prometheus + Alertmanager.
7. Đảm bảo tính idempotent cho toàn bộ pipeline bằng idempotency key lưu trong Database.
8. Kiểm thử toàn bộ luồng từ lúc có comment mới trên Facebook cho đến khi hệ thống nhận webhook, đưa vào Kafka, xử lý, lưu trữ và phản hồi.
9. Kiểm thử các kịch bản lỗi: gửi Facebook thất bại, retry hết lần, message vào DLQ và cảnh báo được kích hoạt.

3 Bài 1: Tích hợp Facebook API và xây dựng Backend

3.1 Mục tiêu

Tích hợp với Facebook Graph API và xây dựng một dịch vụ backend trung gian.

3.2 Yêu cầu

- Tạo Facebook Page và Facebook App
Ví dụ: Tạo một Page giả lập cho cửa hàng hoặc trung tâm đào tạo, sau đó tạo Facebook App để lấy quyền truy cập và cấu hình webhook.
- Lấy Page Access Token
Ví dụ: Sử dụng token để đọc danh sách bài viết của Page hoặc đăng một bài viết thử nghiệm thông qua backend.
- Cài đặt các API:
 - GET /posts
 - POST /post
 - GET /comments
- Backend phải đóng vai trò proxy (frontend không gọi trực tiếp đến Facebook)
Ví dụ: Giao diện web gọi API GET /posts của hệ thống bạn; backend mới là nơi tiếp tục gọi Facebook Graph API.

- Thiết kế cơ chế xác thực và phân quyền phù hợp cho dashboard quản trị
Ví dụ: Chỉ quản trị viên mới được phép ẩn bình luận hoặc gửi phản hồi tự động.
- Chuẩn hóa response API, mã lỗi và thông báo lỗi
Ví dụ: Khi token hết hạn, API trả về mã lỗi rõ ràng như 401 `Unauthorized` kèm thông báo dễ hiểu thay vì lỗi mơ hồ.
- Ghi log đầy đủ cho các request gửi đến Facebook và phản hồi trả về
Ví dụ: Lưu log thời điểm gửi request đăng bài, nội dung rút gọn và mã trạng thái trả về từ Facebook.
- Xử lý lỗi phía Facebook API theo hướng có thể giám sát và khôi phục
Ví dụ: Nếu Facebook tạm thời lỗi 500, hệ thống ghi nhận lỗi, thử lại theo chính sách retry và phát cảnh báo nếu lỗi lặp nhiều lần.

3.3 Kết quả mong đợi

- Các lệnh gọi API hoạt động thành công
- Có thể tạo bài viết thông qua backend
- API có cơ chế xử lý lỗi, log và phản hồi chuẩn hóa

4 Bài 2: Xử lý thời gian thực với Webhook và Kafka

4.1 Mục tiêu

Xây dựng hệ thống hướng sự kiện có khả năng xử lý theo thời gian thực.

4.2 Yêu cầu

- Cài đặt service `webhook-service` (port 3001) để nhận webhook từ Facebook, xác thực request, parse payload và đẩy dữ liệu vào Kafka
Ví dụ: Service này đóng vai trò điểm vào của toàn bộ luồng thời gian thực: nhận event từ Facebook, kiểm tra chữ ký, chuẩn hóa dữ liệu ban đầu rồi publish vào topic `raw_events`.
- Cài đặt Webhook endpoint để nhận sự kiện do Facebook gửi đến
Ví dụ: Khi có bình luận mới vào một bài viết của Page, Facebook gửi HTTP POST đến endpoint `/webhook` của hệ thống.
- Đăng ký nhận sự kiện bình luận từ Facebook
Ví dụ: Chỉ khi subscription được cấu hình đúng, comment mới trên bài viết mới sinh ra webhook event gửi về backend.
- Normalize event về schema chuẩn rồi đưa vào Kafka topic `raw_events`
Ví dụ: Comment và Message có cấu trúc payload khác nhau từ Facebook, sau bước normalize cả hai đều ra cùng một schema để Core Service xử lý.

- Cài đặt Core Service consume `raw_events` và xử lý theo pipeline:
 - Phát hiện spam (chứa liên kết, lặp nội dung nhiều lần)
 - Xác định ý định người dùng (intent) và cảm xúc (sentiment) bằng AI
Ví dụ: “Shop ơi giá bao nhiêu?” → intent: hỏi giá, sentiment: trung tính; “Mình chưa nhận được hàng” → intent: khiếu nại / hỗ trợ, sentiment: tiêu cực; “Bài viết hay quá” → intent: khen / tương tác tích cực, sentiment: tích cực.
Gợi ý: Có thể dùng API của **OpenAI**, **Gemini**, **Claude**, **Grok** hoặc mô hình tương đương, nên thiết kế prompt ngắn gọn, kết quả trả về JSON để dễ parse và retry.
 - Ra quyết định tự động dựa trên kết quả phân loại, ví dụ:
 - * Spam nhẹ → ẩn bình luận ngay.
 - * Spam lặp lại 3 lần trong 24 giờ → đưa người dùng vào blacklist nội bộ và không gửi auto reply nữa.
 - * Link độc hại, scam hoặc bot rõ ràng → ẩn ngay và đẩy sang hàng chờ để quản trị viên review thủ công.
 - * Tài khoản tái phạm nhiều lần → quản trị viên block thủ công trên Facebook Page, hoặc chỉ thử block qua API nếu đã kiểm chứng thử nghiệm gọi API block user comment thành công và xác nhận endpoint hoạt động ổn định.
- Thiết kế consumer chịu được tải tăng đột biến và tránh mất dữ liệu
Ví dụ: Khi một bài viết viral có hàng nghìn bình luận trong vài phút, consumer vẫn phải xử lý dần mà không bỏ sót event.
- Xây dựng cơ chế theo dõi trạng thái xử lý từng sự kiện
Ví dụ: Một comment có thể đi qua các trạng thái như `received`, `processed`, `replied`, `failed`.
- Ghi nhận và phân tích các trường hợp xử lý thất bại
Ví dụ: Nếu gọi Facebook API bị timeout, hệ thống publish `send_failed` và Retry Service sẽ xử lý tiếp.

4.3 Logic xử lý lỗi

- Áp dụng rate limiting
Ví dụ: Một tài khoản gửi 20 bình luận trong 1 phút có thể bị đánh dấu bất thường. Hệ thống vẫn ghi nhận các bình luận này nhưng tạm thời không tự động gọi AI, không tự động phản hồi và không áp dụng automation rule. Thay vào đó, sự kiện được chuyển sang trạng thái `pending_review` để kiểm tra thêm.
- Retry Service thực hiện tối đa N lần với exponential backoff
Ví dụ: Lần 1 chờ 1 giây, lần 2 chờ 2 giây, lần 3 chờ 4 giây. Sau N lần thất bại, message được chuyển sang topic `dead_letter`.
- Áp dụng circuit breaker khi dịch vụ downstream lỗi liên tiếp
Ví dụ: Nếu 10 request liên tiếp tới Facebook API đều thất bại, hệ thống tạm ngắt gọi trong một khoảng thời gian ngắn.

- Đảm bảo consumer có tính idempotent để không xử lý trùng sự kiện
Ví dụ: Nếu cùng một command event bị consume lại 2 lần, Backend API kiểm tra idempotency key trong Database và bỏ qua lần thứ hai.
- Dead Letter Queue và cảnh báo
Ví dụ: Message vào topic `dead_letter` kích hoạt Prometheus alert, Alertmanager bắn thông báo Slack ngay lập tức để nhóm vận hành xử lý thủ công.

4.4 Kết quả mong đợi

- Sự kiện được truyền qua Kafka thành công
- Dữ liệu được lưu trữ và phân loại
- Hệ thống thể hiện rõ cơ chế retry, circuit breaker và idempotent trong luồng xử lý
- Message thất bại được chuyển sang Dead Letter Queue và kích hoạt cảnh báo

5 Bài 3: Phân tích cảm xúc bằng AI và tự động hóa

5.1 Mục tiêu

Tích hợp AI và xây dựng hệ thống tự động hóa phản hồi.

Giải thích: Sau khi dữ liệu bình luận đã được thu thập và làm sạch, bước tiếp theo là “hiểu” nội dung của bình luận. Đây là lúc AI hỗ trợ xác định cảm xúc để hệ thống có thể tự động đưa ra hành động phù hợp.

5.2 Yêu cầu

- Cài đặt chức năng phân tích cảm xúc:
Ví dụ: Bình luận *Dịch vụ rất tốt, mình sẽ quay lại* có thể được phân loại là tích cực; *Sản phẩm tạm ổn* là trung tính; *Trải nghiệm quá tệ* là tiêu cực.
 - Tích cực
 - Trung tính
 - Tiêu cực
- Xây dựng các luật tự động hóa:
 - Tích cực → Cảm ơn người dùng
Ví dụ: Với comment *Shop hỗ trợ rất nhanh*, hệ thống có thể trả lời *Cảm ơn bạn đã ủng hộ shop!*
 - Tiêu cực → Xin lỗi người dùng
Ví dụ: Với comment *Mình chờ quá lâu*, hệ thống có thể phản hồi *Rất xin lỗi vì trải nghiệm chưa tốt, bên mình sẽ kiểm tra ngay.*
 - Spam → Ẩn bình luận
Ví dụ: Comment quảng cáo lặp lại nhiều lần hoặc chứa link lạ có thể bị hệ thống tự động ẩn.

5.3 Yêu cầu bắt buộc có chấm điểm

- Cơ chế retry với exponential backoff
Ví dụ: Khi gọi Facebook API bị timeout, Retry Service thử lại sau 1 giây, 2 giây và 4 giây trước khi chuyển sang Dead Letter Queue.
- Mẫu thiết kế circuit breaker
Ví dụ: Nếu Facebook API phản hồi lỗi liên tục, breaker trong service **backend-api** chuyển sang trạng thái **open** để tránh tiếp tục gửi request vô ích; nếu **core-service** gọi AI bên ngoài thì cũng có thể áp dụng tương tự ở đó.
- Kafka consumer có tính idempotent
Ví dụ: Mỗi **command_id** được lưu vào Database sau khi xử lý; nếu **command_id** đã tồn tại thì bỏ qua, không gửi reply lần thứ hai.
- Dead Letter Queue và cảnh báo vận hành
Ví dụ: Message vào topic **dead_letter** kích hoạt Prometheus alert và Alertmanager gửi thông báo Slack ngay lập tức.

Lưu ý: Đây không còn là phần tùy chọn mà là yêu cầu bắt buộc và có chấm điểm riêng. Retry giúp xử lý lỗi tạm thời, circuit breaker giúp tránh gọi lặp vào một dịch vụ đang lỗi, idempotent giúp một sự kiện dù bị xử lý lặp cũng không làm dữ liệu sai hoặc bị duplicate, Dead Letter Queue giúp không mất message và có thể xử lý thủ công khi cần. Khi trình bày bài làm, sinh viên cần chỉ ra rõ mình hiện thực bốn cơ chế này ở đâu, hoạt động thế nào và được kiểm chứng bằng kịch bản nào.

5.4 Tiêu chí chấm điểm

- **Retry:** Có chiến lược thử lại rõ ràng với exponential backoff, giới hạn số lần thử, và phân biệt lỗi tạm thời với lỗi không thể khôi phục
Ví dụ: Timeout mạng có thể retry, nhưng lỗi **invalid token** thì không nên retry vô hạn.
- **Circuit Breaker:** Có điều kiện đóng/mở rõ ràng và tránh gọi liên tục vào dịch vụ đang lỗi
Ví dụ: Sau 5 lỗi liên tiếp thì mở mạch, chờ 30 giây rồi thử lại ở trạng thái **half-open**.
- **Idempotent:** Có cơ chế nhận diện hoặc loại bỏ sự kiện trùng để tránh ghi trùng dữ liệu hay phản hồi lặp
Ví dụ: Nếu cùng một **command_id** được consume lại, Database đã có key đó và Backend API bỏ qua, không gửi reply lần thứ hai.
- **Dead Letter Queue:** Có cơ chế lưu message thất bại và cảnh báo vận hành kịp thời
Ví dụ: Message vào topic **dead_letter** được Prometheus phát hiện qua offset tăng và Alertmanager bắn alert trong vòng dưới 1 phút.

5.5 Kết quả mong đợi

- Bình luận được phân loại theo cảm xúc

- Hệ thống tạo phản hồi tự động
- Luồng lỗi hoạt động đúng: retry → dead letter → cảnh báo

Sinh viên cần lưu ý: Đừng chỉ trình bày kết quả phân loại. Hãy mô tả rõ vì sao hệ thống chọn hành động đó, ví dụ: bình luận tiêu cực thì gửi lời xin lỗi, spam thì ẩn bình luận. Đồng thời mình chứng được rằng khi gửi Facebook thất bại, hệ thống không mất message mà xử lý đúng theo pipeline retry và DLQ.

A Phụ lục: Cài đặt môi trường phát triển bằng Docker

A.1 Mục tiêu

Phụ lục này giúp sinh viên dựng môi trường chạy Kafka và PostgreSQL trên máy cá nhân bằng Docker để phục vụ việc phát triển, kiểm thử và demo hệ thống.

A.2 Sinh viên cần cài đặt trước

- Docker
- Docker Compose plugin (thường đi kèm Docker Desktop hoặc Docker Engine mới)

Ví dụ: Sau khi cài xong, sinh viên có thể kiểm tra bằng lệnh `docker -version` và `docker compose version`.

A.3 Cấu trúc thư mục

Trước khi tạo file `docker-compose.yml`, sinh viên cần tạo cấu trúc thư mục sau để chứa các file cấu hình:

```
fb_api/
  docker-compose.yml
prometheus/
  prometheus.yml
  alert.rules.yml
alertmanager/
  alertmanager.yml
services/
  backend-api/
  webhook-service/
  core-service/
  retry-service/
```

A.4 File `docker-compose.yml` mẫu

Sinh viên có thể tạo file `docker-compose.yml` với nội dung như sau:

```

services:

# -- Ha tang Kafka --
zookeeper:
  image: confluentinc/cp-zookeeper:7.6.1
  container_name: fb_api-zookeeper
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000
  ports:
    - "2181:2181"

kafka:
  image: confluentinc/cp-kafka:7.6.1
  container_name: fb_api-kafka
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

# -- Kafka UI --
kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: fb_api-kafka-ui
  depends_on:
    - kafka
  ports:
    - "8080:8080"
  environment:
    KAFKA_CLUSTERS_0_NAME: fb_api-local
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
    KAFKA_CLUSTERS_0_ZOOKEEPER: zookeeper:2181

# -- Kafka Exporter (bridge Kafka -> Prometheus) --
kafka-exporter:
  image: danielqsj/kafka-exporter:latest
  container_name: fb_api-kafka-exporter
  depends_on:
    - kafka
  ports:
    - "9308:9308"
  command:
    - "--kafka.server=kafka:9092"

# -- Prometheus --

```

```

prometheus:
  image: prom/prometheus:latest
  container_name: fb_api-prometheus
  depends_on:
    - kafka-exporter
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - ./prometheus/alert.rules.yml:/etc/prometheus/alert.rules.yml
    - prometheus_data:/prometheus
  command:
    - "--config.file=/etc/prometheus/prometheus.yml"
    - "--storage.tsdb.path=/prometheus"

# -- Alertmanager --
alertmanager:
  image: prom/alertmanager:latest
  container_name: fb_api-alertmanager
  depends_on:
    - prometheus
  ports:
    - "9093:9093"
  volumes:
    - ./alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager.yml
  command:
    - "--config.file=/etc/alertmanager/alertmanager.yml"

# -- PostgreSQL --
postgres:
  image: postgres:16
  container_name: fb_api-postgres
  environment:
    POSTGRES_DB: fb_api_db
    POSTGRES_USER: fb_api_user
    POSTGRES_PASSWORD: fb_api_password
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
  prometheus_data:

```

Giải thích các thành phần mới:

- **kafka-ui** (port 8080): Giao diện web để xem topic, xem nội dung message, theo dõi consumer group và đọc message trong topic `dead_letter` khi cần xử lý thủ công.
- **kafka-exporter** (port 9308): Kết nối vào Kafka broker và expose các metric (consumer lag, offset, throughput) dưới dạng HTTP endpoint để Prometheus

scrape. Đây là cầu nối giữa Kafka và Prometheus.

- **prometheus** (port 9090): Scrape metric từ kafka-exporter mỗi 15 giây, lưu lại theo thời gian và đánh giá alert rule.
- **alertmanager** (port 9093): Nhận alert từ Prometheus và route đến kênh Slack hoặc email đã cấu hình.
- **prometheus_data**: Volume lưu dữ liệu metric lâu dài, không bị mất khi container restart.

A.5 Cấu hình Prometheus (prometheus/prometheus.yml)

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - alertmanager:9093

rule_files:
  - "alert.rules.yml"

scrape_configs:
  # Scrape metric của chính Prometheus
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  # Scrape metric Kafka từ kafka-exporter
  - job_name: "kafka"
    static_configs:
      - targets: ["kafka-exporter:9308"]
```

Giải thích: `scrape_interval: 15s` nghĩa là Prometheus hỏi kafka-exporter mỗi 15 giây một lần. Mục `alerting` chỉ cho Prometheus biết Alertmanager đang chạy ở đâu để gửi alert đến.

A.6 Cấu hình Alert Rule (prometheus/alert.rules.yml)

```
groups:
  - name: kafka_alerts
    rules:

      # Cảnh báo khi có message mới vào Dead Letter Queue
      - alert: DeadLetterQueueReceived
```

```

expr: >
  increase(
    kafka_topic_partition_current_offset{
      topic="dead_letter"
    }[1m]
  ) > 0
for: 0m
labels:
  severity: critical
annotations:
  summary: "Có message mới vào Dead Letter Queue"
  description: >
    Topic dead_letter vừa nhận thêm message.
    Kiểm tra Kafka UI tại http://localhost:8080
    để xem nội dung và xử lý thủ công.

# Cảnh báo khi consumer lag quá cao
- alert: KafkaConsumerLagHigh
  expr: kafka_consumer_group_lag > 500
  for: 2m
  labels:
    severity: warning
  annotations:
    summary: "Consumer lag cao bất thường"
    description: >
      Consumer group {{ $labels.consumer_group }}
      đang lag {{ $value }} message trên topic
      {{ $labels.topic }}.

# Cảnh báo khi không có event nào vào raw_events
- alert: WebhookReceiverSilent
  expr: >
    increase(
      kafka_topic_partition_current_offset{
        topic="raw_events"
      }[5m]
    ) == 0
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "Không có event nào từ Facebook trong 10 phút"
    description: >
      Topic raw_events không có message mới.
      Kiểm tra lại Facebook Webhook subscription.

```

Giải thích:

- `increase(...[1m]) > 0`: Kiểm tra xem offset có tăng trong 1 phút vừa rồi không.

Nếu có nghĩa là có message mới vào topic, kích hoạt alert ngay (for: 0m).

- `kafka_consumer_group_lag > 500`: Nếu consumer đang tồn đọng hơn 500 message chưa xử lý trong 2 phút liên tiếp thì cảnh báo.
- `WebhookReceiverSilent`: Nếu không có event nào từ Facebook trong 10 phút thì có thể webhook đã bị ngắt kết nối.

A.7 Cấu hình Alertmanager (alertmanager/alertmanager.yml)

```
global:
  slack_api_url: "https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK"

route:
  receiver: "slack-notifications"
  group_wait: 10s
  group_interval: 5m
  repeat_interval: 1h
  routes:
    - match:
        severity: critical
      receiver: "slack-notifications"
      group_wait: 0s

receivers:
  - name: "slack-notifications"
    slack_configs:
      - channel: "#fb-api-alerts"
        send_resolved: true
        title: >-
          [{{ .Status | toUpper }}] [{{ .GroupLabels.alertname }}]
        text: >-
          {{ range .Alerts }}
            [{{ .Annotations.description }}]
          {{ end }}
```

Giải thích:

- `group_wait: 0s` cho alert `critical`: Alert về Dead Letter Queue được gửi ngay lập tức, không chờ gom nhóm.
- `repeat_interval: 1h`: Không spam Slack, nếu lỗi vẫn còn sau 1 giờ mới gửi lại.
- `send_resolved: true`: Khi vấn đề được giải quyết, Slack nhận thêm thông báo “đã xử lý xong”.
- Sinh viên cần thay `YOUR/SLACK/WEBHOOK` bằng Incoming Webhook URL thật từ cài đặt Slack workspace.

A.8 Các bước khởi động môi trường

1. Tạo cấu trúc thư mục và các file cấu hình như mô tả ở trên.
2. Chạy lệnh `docker compose up -d` để khởi động tất cả container.
3. Kiểm tra container đang chạy bằng lệnh `docker compose ps`.
4. Truy cập Kafka UI tại `http://localhost:8080` để xem topic và message.
5. Truy cập Prometheus tại `http://localhost:9090` để kiểm tra metric và alert rule.
6. Truy cập Alertmanager tại `http://localhost:9093` để kiểm tra trạng thái alert.
7. Kiểm tra log nếu có lỗi bằng lệnh `docker compose logs <tên-container>`.

Kiểm tra kết nối Kafka – Prometheus: Sau khi khởi động, truy cập `http://localhost:9308/metrics` để xem các metric mà kafka-exporter đang expose. Tìm metric `kafka_topic_partition_current_offset` để xác nhận kafka-exporter đã kết nối vào Kafka thành công. Sau đó mở `http://localhost:9090/targets`; mục `kafka` phải có trạng thái UP.

A.9 Tạo Kafka topic mẫu

Sau khi Kafka chạy, sinh viên có thể tạo các topic cần thiết bằng lệnh:

```
docker exec -it fb_api-kafka kafka-topics \
  --create --topic raw_events \
  --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1
```

```
docker exec -it fb_api-kafka kafka-topics \
  --create --topic reply_commands \
  --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1
```

```
docker exec -it fb_api-kafka kafka-topics \
  --create --topic send_failed \
  --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1
```

```
docker exec -it fb_api-kafka kafka-topics \
  --create --topic send_retry \
  --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1
```

```
docker exec -it fb_api-kafka kafka-topics \
  --create --topic dead_letter \
  --bootstrap-server localhost:9092 \
  --partitions 1 --replication-factor 1
```

A.10 Kiểm tra PostgreSQL

Sinh viên có thể truy cập PostgreSQL bằng lệnh:

```
docker exec -it fb_api-postgres psql -U fb_api_user -d fb_api_db
```

Sau đó có thể tạo bảng idempotency key:

```
CREATE TABLE idempotency_keys (  
  command_id VARCHAR(100) PRIMARY KEY,  
  processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE comments (  
  id SERIAL PRIMARY KEY,  
  comment_id VARCHAR(100) UNIQUE NOT NULL,  
  post_id VARCHAR(100) NOT NULL,  
  message TEXT,  
  intent VARCHAR(50),  
  sentiment VARCHAR(20),  
  status VARCHAR(20) DEFAULT 'received',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Ví dụ: Bảng `idempotency_keys` được dùng bởi Backend API để kiểm tra xem một `command_id` đã được xử lý chưa trước khi gọi Facebook Graph API. Bảng `comments` lưu toàn bộ lịch sử xử lý của từng bình luận.

A.11 Cách dừng môi trường

- Dừng container: `docker compose down`
- Dừng và xóa cả volume dữ liệu: `docker compose down -v`